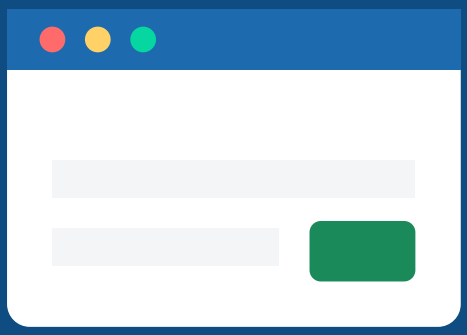

How to Start a Windows Desktop Application

A beginner-friendly guide using C#, .NET, WinForms and WPF

Install the tools • Build your first app • Publish it

Updated October 2026

Contents

1. Choose your technology
2. Install the tools
3. Your first app: Windows Forms
4. Your first app: WPF
5. Understanding the project files
6. Build, run and publish
7. Common mistakes and next steps

> **Who is this for?**
> Complete beginners who want to build a program with windows, buttons and text boxes that runs on Windows. Basic familiarity with any programming language helps, but is not required.

What you will need

- A Windows 10 or 11 PC
- About 10 GB of free disk space
- An internet connection for the one-time download
- 30 to 60 minutes

1. Choose your technology

Windows offers several ways to build desktop apps. For a first project, the best combination is **C#** with the free **.NET** platform. You then pick a UI framework:

Framework	Best for	Difficulty	Notes
Windows Forms (WinForms)	Simple tools, business apps, learning	Easy	Drag-and-drop designer. Fastest way to see results.
WPF	Modern, flexible, good-looking interfaces	Medium	Uses XAML markup for the layout. Strong data binding.
WinUI 3	Newest Windows 11 look and feel	Medium to hard	Modern, but fewer tutorials and a heavier setup.
.NET MAUI	One app for Windows, Android, iOS, Mac	Medium	Choose it if you need cross-platform.

Recommendation
Start with **Windows Forms** to learn the basics quickly, then move to **WPF** when you want more control over the design. Both are covered in this guide.

Other languages

You can also build Windows apps with Python (Tkinter, PyQt), Java (JavaFX), or JavaScript (Electron). C# is the most natural fit for Windows because Visual Studio gives you a complete designer, debugger and project system out of the box.

2. Install the tools

You only need one program: **Visual Studio Community**. It is free for students, open-source developers and individuals.

Step by step

- **Download** Visual Studio Community from visualstudio.microsoft.com.
- Run the installer. When asked to choose **Workloads**, tick **.NET desktop development**.
- Click **Install** and wait. The download is several GB, so it can take a while.
- Launch Visual Studio and sign in with a Microsoft account (or skip for now).

> **Prefer the command line?**
> Install the free **.NET SDK** from dotnet.microsoft.com and use any editor, such as VS Code. Check it works by opening a terminal and typing:

```
dotnet --version
```

If a version number appears (for example 8.x or 9.x), you are ready.

3. Your first app: Windows Forms

We will build a small app with a button that counts your clicks.

Create the project

- Open Visual Studio and click **Create a new project**.
- Search for **Windows Forms App** (choose the C# one, *not* "(.NET Framework)" unless you need old compatibility).
- Click **Next**, name it **MyFirstApp**, choose a folder, and click **Next**.
- Choose the latest .NET version offered and click **Create**.

Design the window

- In **Solution Explorer**, double-click **Form1.cs** to open the designer.
- Open the **Toolbox** (View menu) and drag a **Button** and a **Label** onto the form.
- Click the button, then in the **Properties** panel change **Text** to "Click me".
- Double-click the button. Visual Studio creates a click handler and opens the code.

Add the code

Inside the form class, add a counter and fill in the handler. The method name will match your control names (`button1`, `label1`).

```
public partial class Form1 : Form
{
    private int count = 0;

    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        count++;
        label1.Text = $"You clicked {count} times";
    }
}
```

Run it

Press **F5** (or the green Start button). Your window appears. Click the button and watch the label change. Congratulations, you have built a Windows app!

4. Your first app: WPF

WPF separates the **look** (XAML) from the **logic** (C#). Create a project the same way, but choose **WPF Application**.

MainWindow.xaml (the layout)

```
<Window x:Class="MyWpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="My First WPF App" Height="220" Width="340">
    <StackPanel Margin="20">
        <TextBlock Text="Enter your name:" />
        <TextBox x:Name="NameBox" Margin="0,5,0,0" />
        <Button Content="Say Hello" Margin="0,10,0,0"
                Click="Hello_Click" />
        <TextBlock x:Name="Output" Margin="0,10,0,0"
                   FontSize="16" />
    </StackPanel>
</Window>
```

MainWindow.xaml.cs (the logic)

```
private void Hello_Click(object sender, RoutedEventArgs e)
{
    Output.Text = $"Hello, {NameBox.Text}!";
}
```

Press **F5** to run it. Type your name, click the button, and the greeting appears.

> **Tip: layout containers**
> WPF arranges controls with containers such as **StackPanel** (a line), **Grid** (rows and columns) and **DockPanel** (edges). Learn **Grid** first, as it is the most useful.

Creating projects from the command line

```
dotnet new winforms -n MyFirstApp
dotnet new wpf -n MyWpfApp
cd MyWpfApp
dotnet run
```

5. Understanding the project files

A new project contains several files. Here is what the important ones do:

File	Purpose
`Program.cs`	The entry point. Starts the application and shows the first window (WinForms).
`Form1.cs`	Your code for the window: button clicks, logic, and so on (WinForms).
`Form1.Designer.cs`	Auto-generated by the visual designer. Avoid editing by hand.
`MainWindow.xaml`	The layout and controls of the window (WPF).
`MainWindow.xaml.cs`	The code behind the window (WPF).
`App.xaml`	Application-wide settings and styles (WPF).
`*.csproj`	Project settings: .NET version, packages, output type.
`bin/ and obj/`	Build output. Created automatically; safe to delete.

Useful concepts to learn next

- **Events**: code that runs when something happens (a click, a key press, a window closing).
- **Controls**: Button, TextBox, Label, ComboBox, CheckBox, ListBox, DataGridView / DataGrid.
- **Layout**: anchoring and docking (WinForms), Grid and panels (WPF).
- **Data**: reading and saving files (JSON, CSV) or a database (SQLite is free and simple).
- **MVVM**: a design pattern that keeps WPF apps organized as they grow.

6. Build, run and publish

Debug vs Release

While developing, run with **F5** (Debug mode). When you are ready to share the app, build a **Release** version, which is faster and has no debug data.

Publish with Visual Studio

- Right-click the project in Solution Explorer and choose **Publish**.
- Choose **Folder** as the target and click **Next**, then **Finish**.
- Click **Publish**. Visual Studio produces the output in the folder shown.

Publish with the command line

```
dotnet publish -c Release -r win-x64 --self-contained true -p:PublishSingleFile=true
```

This creates a single .exe that runs on Windows without needing .NET to be installed. Look in `bin\Release\net*\win-x64\publish`.

Option	Meaning
`--self-contained true`	Bundles .NET with your app. Bigger file, but runs anywhere on Windows.
`--self-contained false`	Smaller file, but the user must have the .NET runtime installed.
`PublishSingleFile=true`	Packs everything into one .exe file.

Making an installer

To give your app to others as a proper installer, look at **MSIX** (built into Visual Studio) or free tools such as **Inno Setup**.

7. Common mistakes and next steps

Common mistakes

Problem	Fix
Template "Windows Forms App" is missing	Re-run the Visual Studio Installer and add the **.NET desktop development** workload.
The window freezes during a long task	Do slow work in the background with `async/await` so the interface stays responsive.
Editing `Designer.cs` breaks the form	Make changes in the designer or the Properties panel instead.
App will not start on another PC	Publish self-contained, or install the .NET Desktop Runtime on that PC.
Button does nothing when clicked	Check that the Click event is actually connected (Properties panel, lightning-bolt icon).

Practice project ideas

- Calculator
- To-do list that saves to a file
- Unit converter
- Simple notepad
- Expense tracker with SQLite

Free learning resources

- **Microsoft Learn: Windows Forms** – learn.microsoft.com/dotnet/desktop/winforms
- **Microsoft Learn: WPF** – learn.microsoft.com/dotnet/desktop/wpf
- **WPF Tutorial** – wpf-tutorial.com
- **C# Yellow Book** by Rob Miles – free PDF to learn C# itself
- **Syncfusion Succinctly** free ebooks – syncfusion.com/succinctly-free-ebooks

> **Your learning path**
> **1)** Learn basic C# → **2)** Build 3 small WinForms apps → **3)** Learn files and databases → **4)** Move to WPF and MVVM → **5)** Publish and share your work.